

UNIVERSITI TEKNOLOGI MALAYSIA
FACULTY OF COMPUTING

CHAPTER 10 LAB MANUAL

Connecting to Database with PDO

(Refactor the Books REST API to use real MySQL storage)

FIELD	DETAILS
Course	SCSM2223 — Cross-Platform Application Development
Chapter	10 — Connecting to Database (PDO)
Lab time	2 hours
Lab assessment	
Software	Laragon (MySQL + PHP), Composer, HeidiSQL, Sublime Text / VS Code

1. Lab Objectives

By the end of this lab you will be able to:

1. Create a MySQL database and a books table from a schema.sql script.
2. Build a PDO factory that reads its credentials from a .env file.
3. Implement a BookRepository with prepared statements for every CRUD operation.
4. Refactor a Slim controller to delegate persistence to the repository (zero SQL in the controller).
5. Verify that user-supplied input cannot be used to perform SQL injection.
6. Test all six endpoints with the VS Code REST Client (or Postman / curl).

2. Prerequisites

Verify each item before starting.

#	Tool	Verify by running...	Expected result
1	Laragon (Full edition)	Open Laragon, click Start All	Apache + MySQL turn green
2	PHP 8.1+	php -v	PHP 8.1.x or newer
3	Composer	composer --version	Composer 2.x
4	HeidiSQL	Laragon → Database → HeidiSQL	Connection list opens
5	Ch9 solution	Copy Ch9_BooksAPI_Solution to a new folder named books-api-mysql, OR start fresh from a Slim 4 scaffold	Project compiles

TIP

Always use the Laragon Terminal — it pre-loads PHP, Composer, and MySQL onto the PATH.

3. What We Will Build

We will replace Chapter 9's in-memory array with a real MySQL table and a Repository class that owns every SQL statement. The HTTP-facing endpoints stay exactly the same — clients never know the storage changed.

Part A — Set Up the Database

Step 1 — Open HeidiSQL via Laragon

From Laragon, click Database → HeidiSQL. Connect with user root and an empty password (Laragon's default).

Step 2 — Run schema.sql

Open the file sql/schema.sql in HeidiSQL and press F9 to execute, OR run it from the Laragon Terminal:

```
mysql -u root < sql/schema.sql
```

Contents of schema.sql:

```

CREATE DATABASE IF NOT EXISTS books_api
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE books_api;

DROP TABLE IF EXISTS books;

CREATE TABLE books (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  title       VARCHAR(200) NOT NULL,
  author      VARCHAR(150) NOT NULL,
  year        SMALLINT     NOT NULL,
  genre       VARCHAR(80)  NOT NULL DEFAULT 'Uncategorised',
  created_at  DATETIME      NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at  DATETIME      NOT NULL DEFAULT CURRENT_TIMESTAMP
              ON UPDATE CURRENT_TIMESTAMP
) ENGINE=InnoDB;

INSERT INTO books (title, author, year, genre) VALUES
('Clean Code',          'Robert C. Martin', 2008, 'Software Engineering'),
('Eloquent JavaScript', 'Marijn Haverbeke', 2018, 'Programming'),
('Vue.js 3 By Example', 'John Au-Yeung',    2021, 'Web Development');

```

CHECKPOINT After running this, `SELECT * FROM books_api.books` should return three rows.

Part B — Wire the Project to MySQL

Step 3 — Add phpdotenv

phpdotenv reads a `.env` file and exposes its variables on `$_ENV`. Install it with Composer:

```
composer require vlucas/phpdotenv
```

Step 4 — Create `.env` and `.env.example`

Create two files at the project root. Commit only `.env.example` to git.

```

# .env.example (and copy to .env)
DB_HOST=127.0.0.1
DB_PORT=3306
DB_NAME=books_api
DB_USER=root
DB_PASS=
DB_CHARSET=utf8mb4
APP_DEBUG=true

```

Step 5 — `src/Database.php` (PDO factory)

```

<?php
namespace App;

use PDO;
use PDOException;

final class Database

```

```

{
    private static ?PDO $pdo = null;

    public static function get(): PDO
    {
        if (self::$pdo) return self::$pdo;

        $dsn = sprintf(
            'mysql:host=%s;port=%s;dbname=%s;charset=%s',
            $_ENV['DB_HOST'] ?? '127.0.0.1',
            $_ENV['DB_PORT'] ?? '3306',
            $_ENV['DB_NAME'] ?? 'books_api',
            $_ENV['DB_CHARSET'] ?? 'utf8mb4'
        );
        try {
            self::$pdo = new PDO($dsn, $_ENV['DB_USER'] ?? 'root', $_ENV['DB_PASS'] ?? '', [
                PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
                PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
                PDO::ATTR_EMULATE_PREPARES => false,
            ]);
        } catch (PDOException $e) {
            error_log('[DB] ' . $e->getMessage());
            throw new \RuntimeException('Database connection failed', 500, $e);
        }
        return self::$pdo;
    }
}

```

Step 6 — Bootstrap dotenv in public/index.php

```

<?php
use Dotenv\Dotenv;
use Slim\Factory\AppFactory;

require __DIR__ . '/../vendor/autoload.php';

Dotenv::createImmutable(__DIR__ . '/../')->safeLoad();

$app = AppFactory::create();
$app->add(new App\Middleware\JsonBodyParser());
$app->add(new App\Middleware\Cors());
$app->addRoutingMiddleware();
$app->addErrorMiddleware(true, true, true);
(require __DIR__ . '/../src/routes.php')($app);
$app->run();

```

Part C — Build the BookRepository

Create src/Repositories/BookRepository.php. Every database operation lives here. Notice that user input is ALWAYS passed via :placeholders — never concatenated into the SQL string.

Step 7 — Read methods

```
<?php
namespace App\Repositories;
use PDO;

final class BookRepository
{
    public function __construct(private PDO $pdo) {}

    public function all(string $q = '', int $limit = 0): array {
        $sql = 'SELECT * FROM books';
        $args = [];
        if ($q !== '') {
            // Each named placeholder must be UNIQUE when
            // ATTR_EMULATE_PREPARES is false (see callout below).
            $sql .= ' WHERE title LIKE :q_title OR author LIKE :q_author';
            $args[':q_title'] = '%' . $q . '%';
            $args[':q_author'] = '%' . $q . '%';
        }
        $sql .= ' ORDER BY id ASC';
        if ($limit > 0) $sql .= ' LIMIT ' . max(1, $limit);
        $stmt = $this->pdo->prepare($sql);
        $stmt->execute($args);
        return $stmt->fetchAll();
    }

    public function find(int $id): ?array {
        $stmt = $this->pdo->prepare('SELECT * FROM books WHERE id = :id');
        $stmt->execute([':id' => $id]);
        $row = $stmt->fetch();
        return $row === false ? null : $row;
    }
}
```

GOTCHA

When `ATTR_EMULATE_PREPARES` is false (Chapter 10's secure default), MySQL real prepared statements treat every named placeholder as DISTINCT. You cannot reuse the same placeholder twice in one query — doing so raises SQLSTATE HY093 "Invalid parameter number". Rule of thumb: each `:name` appears exactly once. If you need the same value in two spots (e.g. title LIKE :q OR author LIKE :q), give them different names (`:q_title`, `:q_author`) and bind both.

Step 8 — Write methods

```
public function create(array $b): int {
    $sql = 'INSERT INTO books (title, author, year, genre)
           VALUES (:title, :author, :year, :genre)';
    $this->pdo->prepare($sql)->execute([
        ':title' => trim($b['title']), ':author' => trim($b['author']),
        ':year' => (int)$b['year'],      ':genre' => trim($b['genre']) ?? 'Uncategorised',
    ]);
    return (int)$this->pdo->lastInsertId();
}

public function update(int $id, array $b): int {
    $sets = []; $args = [':id' => $id];
```

```

        foreach (['title','author','genre'] as $f) {
            if (array_key_exists($f, $b)) { $sets[] = "$f=:f"; $args[":$f"] = trim($b[$f]); }
        }
        if (array_key_exists('year', $b)) { $sets[] = 'year=:year'; $args[':year'] =
(int)$b['year']; }
        if (!$sets) return 0;
        $sql = 'UPDATE books SET ' . implode(', ', $sets) . ' WHERE id=:id';
        $stmt = $this->pdo->prepare($sql);
        $stmt->execute($args);
        return $stmt->rowCount();
    }

    public function delete(int $id): bool {
        $stmt = $this->pdo->prepare('DELETE FROM books WHERE id = :id');
        $stmt->execute([':id' => $id]);
        return $stmt->rowCount() === 1;
    }
}

```

Part D — Refactor the Controller

Open `src/Controllers/BookController.php` and replace its body with the version below. Notice the constructor now receives a `BookRepository` and there is NO SQL anywhere in this file.

Step 9 — BookController (HTTP-only)

```

<?php
namespace App\Controllers;
use App\Repositories\BookRepository;
use Psr\Http\Message\ResponseInterface as Response;
use Psr\Http\Message\ServerRequestInterface as Request;

final class BookController {
    public function __construct(private BookRepository $books) {}

    public function index(Request $r, Response $s): Response {
        $p = $r->getQueryParams();
        $rows = $this->books->all((string)($p['q'] ?? ''), (int)($p['limit'] ?? 0));
        return $this->json($s, ['count'=>count($rows), 'data'=>$rows]);
    }

    public function show(Request $r, Response $s, array $a): Response {
        $book = $this->books->find((int)$a['id']);
        return $book ? $this->json($s, $book)
            : $this->json($s, ['error'=>'not found'], 404);
    }

    public function create(Request $r, Response $s): Response {
        $body = (array)$r->getParsedBody();
        $errors = $this->validate($body, true);
        if ($errors) return $this->json($s, ['errors'=>$errors], 400);
        $id = $this->books->create($body);
        return $this->json($s, ['message'=>'Book created', 'data'=>$this->books->find($id)],
201)
            ->withHeader('Location', '/api/books/' . $id);
    }
}

```

```

/* update() and delete() follow the same pattern – see solution */
private function validate(array $b, bool $requireAll): array { /* ... */ }
private function json(Response $r, $data, int $code=200): Response {
    $r->getBody()->write(json_encode($data, JSON_PRETTY_PRINT));
    return $r->withHeader('Content-Type', 'application/json')->withStatus($code);
}
}

```

Step 10 — Wire the controller in src/routes.php

```

<?php
use App\Controllers\BookController;
use App\Database;
use App\Repositories\BookRepository;
use Slim\App;

return function (App $app): void {
    $controller = new BookController(new BookRepository(Database::get()));

    $app->group('/api', function ($g) use ($controller) {
        $g->get ('/books', [$controller, 'index']);
        $g->get ('/books/{id}', [$controller, 'show']);
        $g->post ('/books', [$controller, 'create']);
        $g->put ('/books/{id}', [$controller, 'update']);
        $g->delete('/books/{id}', [$controller, 'delete']);
    });
};

```

Part E — Test the API

Restart the server (Ctrl+C, then `php -S localhost:8000 -t public`).

Test 1 — list and seed data

```

GET http://localhost:8000/api/books
→ 200, three books from the database

```

Test 2 — search and limit

```

GET http://localhost:8000/api/books?q=clean      → 200, only "Clean Code"
GET http://localhost:8000/api/books?limit=2      → 200, first two rows

```

Test 3 — create

```

POST http://localhost:8000/api/books
Content-Type: application/json

{ "title": "Refactoring", "author": "Martin Fowler", "year": 2018 }

Expected: 201 Created, Location: /api/books/4

```

Now open HeidiSQL and run `SELECT * FROM books` — the new row should be there. Persistence is real now.

Test 4 — partial update + delete

```

PUT    http://localhost:8000/api/books/1 { "genre": "Software Craftsmanship" } → 200
DELETE http://localhost:8000/api/books/2                                     → 200

```

Test 5 — restart the server, data persists

Stop the PHP server (Ctrl+C) and start it again. GET /api/books — the changes from Test 4 are still there. Compare this with Chapter 9 where data reset on every request.

Part F — Try (and Fail) at SQL Injection

These attempts demonstrate that prepared statements protect us automatically:

Attempt 1 — id with OR injection

```
GET http://localhost:8000/api/books/1%20OR%201=1 → 404 Book 0 not found
```

Why it fails: the controller casts the path argument to int (the value becomes 1), and even if it didn't, the value is sent through a placeholder.

Attempt 2 — quote escape in q parameter

```
GET http://localhost:8000/api/books?q='%20OR%20'1'='1 → 200, but data: []
```

Why it fails: the apostrophes are sent as DATA bound to :q, so MySQL searches literally for the string ' OR '1'='1 — which matches no titles.

LESSON Never trust input. Always use prepared statements, even for queries you 'know' are safe.

Part G — Self-Check & Extension Tasks

7. Insert a row through HeidiSQL, then GET it via the API.
8. Update a row through the API, then read it back in HeidiSQL.
9. Confirm that POST without title/author/year returns 400 with a per-field error map.
10. Confirm that GET /api/books/9999 returns 404 — not 500.
11. Run an injection attempt of your choice and observe that no rows leak.
12. BONUS — wrap a simulated 'transfer two book copies' update inside a transaction (beginTransaction, commit, rollBack).
13. BONUS — add a count() endpoint that returns the total row count using fetchColumn().
14. BONUS — add an updated_at timestamp to the JSON response and verify it changes after a PUT.

Appendix — Common Errors & Troubleshooting

Symptom	Likely cause	Fix
SQLSTATE[HY000] [2002] No connection	MySQL not running	Start Laragon, click Start All.
SQLSTATE[28000] Access denied for user...	.env credentials wrong	Check DB_USER and DB_PASS in .env.
SQLSTATE[42S02] Base table not found	Schema not loaded	Run sql/schema.sql in HeidiSQL.

Symptom	Likely cause	Fix
PDOException: could not find driver	pdo_mysql extension not loaded	In php.ini uncomment extension=pdo_mysql; verify with php -m findstr pdo_mysql; restart the server. Also confirm php -v matches the version with the extension enabled.
SQLSTATE[HY093] Invalid parameter number	Reused the same named placeholder twice with EMULATE_PREPARES=false	Give each placeholder a unique name (e.g. :q_title and :q_author) and bind both. See the Step 7 callout.
UTF-8 characters look corrupted	Charset not set	Set DB_CHARSET=utf8mb4 in .env and re-run schema.
lastInsertId() returns 0	PDO got reset	Reuse the same PDO instance (Database::get() does this).
Class App\Database not found	Autoload not regenerated	Run composer dump-autoload.
Errors leaked in JSON response	APP_DEBUG=true in production	Set APP_DEBUG=false; log details server-side instead.

End of Lab Manual.